PROJECT REPORT ON

MEDISHARE

MEDICINE DONATION AND DISTRIBUTION PLATFORM

PROJECT REPORT & TECHNICAL GUIDE

Submitted in partial fulfillment of the requirements for the award of **Diploma in Computer Engineering.**

SUBMITTED BY

MANADI RUPA	24004-CS-098
DASI CHARAN KUMAR	24004-CS-080
VOLLALA SHANMUKHA SAI	24004-CS-117
BEJUGAM RAGHAVI	24004-CS-105
MEDIPELLI SHIVANI	24004-CS-101
DARAVATH BASANTHI	24004-CS-123
BUDDA CHAITANYA	24004-CS-068
NALABOLU SATWIK REDDY	24004-CS-082
BIJILI SAI CHARAN	24004-CS-121

Under the guidance of

S. Ravinder, M.Tech

and Principal

P. Srinivas, M.Tech

DEPARTMENT OF COMPUTER ENGINEERING

GOVERNMENT POLYTECHNIC WARANGAL

Warangal, Telangana • Academic Session 2024–2027

DEPARTMENT OF COMPUTER ENGINEERING

GOVERNMENT POLYTECHNIC WARANGAL

CERTIFICATE

This is to certify that this project report entitled **"MEDISHARE: MEDICINE DONATION AND DISTRIBUTION PLATFORM"** is a bonafide record of the project work carried out by the students listed below, in partial fulfillment of the requirements for the award of Diploma in Computer Engineering, during the academic session 2024–2027.

MANADI RUPA — 24004-CS-098

DASI CHARAN KUMAR — 24004-CS-080

VOLLALA SHANMUKHA SAI — 24004-CS-117

BEJUGAM RAGHAVI — 24004-CS-105

MEDIPELLI SHIVANI — 24004-CS-101

DARAVATH BASANTHI — 24004-CS-123

BUDDA CHAITANYA — 24004-CS-068

NALABOLU SATWIK REDDY — 24004-CS-082

BIJILI SAI CHARAN — 24004-CS-121

The work is submitted for academic review. The system is a web application prototype; its results should be treated as an informational aid and not as a substitute for professional medical or commercial advice.

PROJECT GUIDE	**PRINCIPAL**
S. Ravinder, M.Tech	P. Srinivas, M.Tech

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide, **S. Ravinder, M.Tech**, for his guidance, valuable suggestions and encouragement throughout the development of this project. His support helped us understand the technical and practical challenges involved in building a complete medicine donation and distribution platform.

We are thankful to our Principal, **P. Srinivas, M.Tech**, for providing the opportunity and institutional support to undertake this work. We also thank the faculty members of the Department of Computer Engineering for their assistance and feedback.

We acknowledge the open-source technologies used in the MediShare project, including PHP, PostgreSQL, Supabase, Bootstrap, Razorpay, Leaflet and Playwright. Finally, we thank our families and friends for their encouragement.

PROJECT TEAM

MANADI RUPA	**24004-CS-098**
DASI CHARAN KUMAR	**24004-CS-080**
VOLLALA SHANMUKHA SAI	**24004-CS-117**
BEJUGAM RAGHAVI	**24004-CS-105**
MEDIPELLI SHIVANI	**24004-CS-101**
DARAVATH BASANTHI	**24004-CS-123**
BUDDA CHAITANYA	**24004-CS-068**
NALABOLU SATWIK REDDY	**24004-CS-082**
BIJILI SAI CHARAN	**24004-CS-121**

ABSTRACT

MediShare is a web-based medicine donation and distribution platform that connects verified medicine distributors and pharmacies with non-governmental organizations, medical shops, old-age homes, orphanages and charitable hospitals. A large quantity of usable medicine is wasted every year because distributors hold surplus or near-expiry stock while social organizations struggle to obtain medicines for the people they serve. MediShare addresses this gap with a single, verified marketplace.

The application provides role-based access for administrators, distributors and organizations. Distributors publish stock with batch number, expiry date, price and quantity. Organizations search the store, add medicines to a cart, choose a delivery address and pay through Razorpay (UPI, card or net banking) or cash on delivery. Every order passes through a controlled lifecycle: Pending Payment, Pending, Accepted, Packed, Shipped, Out for Delivery and Delivered. Stock is reserved when an order is placed and restored when it is cancelled or rejected. Expired and out-of-stock medicines are removed automatically and the distributor is notified.

The system is implemented with PHP 8 on Apache, PostgreSQL hosted on Supabase, Bootstrap 5 and vanilla JavaScript. It includes admin verification of documents, in-app notifications with an unread badge, reviews, reports with CSV export, audit logs, an email-OTP password-reset flow and performance optimizations for a hosted database. The platform was tested at the API and browser levels; all functional tests passed and cached read paths respond in milliseconds.

KEYWORDS

Medicine donation • Medicine redistribution • E-commerce • Order management • Razorpay • Supabase • PHP • PostgreSQL

INDEX

CONTENTS	PAGE
Executive Summary	6
1. Introduction	7
2. Problem Statement & Motivation	8
3. Project Scope & Objectives	9
4. System Architecture	10
5. Technical Stack	11
6. Database Design	12
7. Implementation	13
8. Order Lifecycle & Payments	15
9. Security	16
10. Performance Optimization	17
11. Testing & Results	18
12. Limitations & Future Work	19
13. Conclusion & References	20
Appendix — Setup & API Reference	21

Executive Summary

MediShare is a complete web application for redistributing usable medicines from verified distributors to verified social organizations. It replaces informal phone and spreadsheet coordination with a single platform that shows live stock, expiry dates, prices and order status.

The platform has three roles. Distributors manage their medicine inventory. Organizations browse the store, place orders and pay online or choose cash on delivery. Administrators verify accounts, review uploaded documents and monitor reports and audit logs.

The main outcomes of the project are:

- a verified network of distributors and recipient organizations;
- expiry-aware inventory with automatic removal of expired and out-of-stock medicines;
- an order lifecycle with stock reservation, cancellation, rejection and delivery tracking;
- server-verified Razorpay payments with UPI, card and net-banking support, plus cash on delivery;
- in-app notifications with an unread badge;
- reviews, reports with CSV export and a complete audit trail;
- an email-OTP password-reset flow;
- caching and lazy database access that keep cached reads in the millisecond range.

The system was validated with API tests, browser tests and a role-based smoke test. All functional checks passed.

1. Introduction

1.1 Background

Medicines are essential, time-sensitive products. Distributors and pharmacies routinely carry surplus stock, and a portion of it approaches expiry before it can be sold. At the same time, NGOs, old-age homes, orphanages, charitable hospitals and small medical shops often lack the funds and supply network to obtain even basic medicines. Much of the surplus is discarded while genuine demand remains unmet.

Most redistribution today happens through phone calls, messaging applications and personal contacts. These channels do not provide verified participants, live stock information, price transparency, order tracking, payment records or an audit trail. MediShare was built to replace this informal process with a structured digital platform.

1.2 Purpose of the Project

The purpose of the project is to design and implement a reliable, easy-to-use platform where verified distributors can list surplus or near-expiry medicines and verified organizations can order them. The project also demonstrates a complete software development cycle: requirements, design, implementation, testing and deployment.

1.3 Scope of the Report

This report describes the problem, the proposed solution, the system architecture, the database design, the implementation of each module, the security and performance measures, and the testing results. Setup instructions and an API reference are included in the appendix.

2. Problem Statement & Motivation

There is no common, trustworthy platform where verified distributors can list surplus or near-expiry medicines and verified social organizations can order them. The absence of such a platform causes avoidable wastage of usable medicines, difficulty for organizations in finding suppliers, no verification of the parties involved, no record of donations, payments or deliveries, no visibility of expiry dates and stock levels, and manual, error-prone order handling.

The motivation for the project comes from three observations:

- medicine waste is often a logistics and visibility problem rather than a shortage of demand;
- social organizations need a simple ordering experience, not a complex enterprise system;
- trust is essential, so every distributor and organization must be verified before participating.

MediShare addresses these issues with verified accounts, a live store, a controlled order lifecycle, online payments and an audit trail.

3. Project Scope & Objectives

3.1 Objectives

1. Build a verified network of medicine distributors and social organizations.
2. Provide real-time visibility of medicine stock, price, batch number and expiry date.
3. Allow organizations to search, filter, order and pay for medicines online.
4. Track every order through a clear lifecycle with cancellation and rejection support.
5. Notify distributors and organizations about orders, payments, stock and expiry events.
6. Give administrators tools to verify documents, manage users, view reports and audit activity.
7. Protect accounts and business data with authentication, authorization and secure coding practices.
8. Keep the application responsive despite a hosted database by caching read-heavy responses.

3.2 In Scope

Registration and document upload, admin verification, medicine inventory, store search and filters, cart, checkout, Razorpay and cash-on-delivery payments, order status management, cancellation and rejection, notifications, reviews, reports and CSV export, audit logs, profile management, a public map and password reset.

3.3 Out of Scope

Direct patient prescriptions, clinical advice, courier integration, a live SMS gateway, a native mobile application, multi-language support and automated document verification are outside the current scope and are listed as future work.

4. System Architecture

MediShare uses a three-tier architecture. The browser renders static pages and JavaScript modules. The application tier is a set of PHP endpoints running on Apache. The data tier is a PostgreSQL database hosted on Supabase. External services provide payments and email OTP delivery.

```
Browser (HTML + Bootstrap + JavaScript)
        |
        |  JSON over HTTP
        v
Apache + PHP 8 API  --->  Razorpay API (orders, payments)
        |            --->  Supabase Auth (password-reset OTP)
        |
        |  PDO (prepared statements)
        v
PostgreSQL database (Supabase)
```

4.1 Modules

- **Authentication and verification:** registration, login, logout, sessions, document review and account approval.
- **Medicine inventory:** add, edit, remove, stock, expiry, batch, category, price and images.
- **Store and cart:** search, filters, cart persistence, delivery address and geolocation.
- **Order management:** order creation, grouping by distributor, status transitions, cancellation and rejection.
- **Payments:** Razorpay order creation and signature verification, cash on delivery and payment status.
- **Notifications:** in-app notifications, unread badge and periodic refresh.
- **Reviews:** one review per delivered order with rating and comment.
- **Reports and audit:** revenue and order statistics, CSV export and activity log.
- **Password reset:** email OTP, reset token and password update.
- **Profile and map:** profile updates, delivery location and a public map page.

4.2 Design Principles

- Role-based access is enforced on the server, not only in the interface.
- Every write that affects stock runs inside a database transaction.
- Payment confirmation depends on a verified gateway signature.
- Read-heavy responses are cached briefly and invalidated on writes.
- Removed medicines are soft-deleted so historical orders remain intact.

5. Technical Stack

Layer	Technology	Purpose
Front end	HTML5, CSS3, Bootstrap 5.3, Bootstrap Icons	Responsive pages and components
Client logic	Vanilla JavaScript	API calls, cart, dashboard rendering, validation
Back end	PHP 8.2	REST-style JSON endpoints, sessions, business logic
Database	PostgreSQL 15 on Supabase	Persistent relational data
Database access	PDO with prepared statements	Safe, parameterized queries
Payments	Razorpay (test mode)	UPI, card and net-banking payments
OTP email	Supabase Auth	Password-reset one-time passwords
Maps	Leaflet, OpenStreetMap, Google Maps links	Location display and directions
Web server	Apache (XAMPP)	Hosting the PHP application
Version control	Git and GitHub	Source control and collaboration
Testing	PHP cURL tests, Playwright	API and browser validation

5.1 Why This Stack

PHP and PostgreSQL are widely available, well documented and suitable for a data-driven web application. Supabase provides a managed PostgreSQL instance without server administration. Bootstrap keeps the interface consistent and responsive. Razorpay offers a straightforward test-mode integration. Playwright provides reliable end-to-end browser testing.

6. Database Design

The database contains ten tables. The design separates common account data from role-specific profiles and keeps order history independent of current inventory.

Table	Key Data	Purpose
`users`	name, email, password hash, phone, address, city, latitude, longitude, role, org type, verified	Common account data
`distributors`	user_id, license number, license image	Distributor profile
`ngos`	user_id, registration number, registration image	Organization profile
`medicines`	name, category, price, MRP, stock, batch, expiry, shelf life, image, distributor, deleted, low-stock flag	Inventory
`orders`	organization, distributor, delivery address, coordinates, total, status, tracking number, timestamps	Order header
`order_items`	order, medicine, quantity, price	Order line items
`transactions`	order, amount, method, status, gateway reference	Payment record
`notifications`	user, type, text, read status, timestamp	User notifications
`reviews`	order, user, rating, comment	Delivered-order feedback
`audit_logs`	user, action, timestamp	Activity trail

6.1 Relationships

- One user has at most one distributor profile or one organization profile.
- One distributor owns many medicines.
- One organization places many orders; one order belongs to one distributor.
- One order contains many order items and has one transaction.
- One delivered order can have one review.

6.2 Design Notes

Medicine rows use a soft-delete flag because order items reference them. Prices are copied into order items at the time of ordering so later price changes do not alter historical orders. Order totals are computed on the server. Status values are constrained so that only valid lifecycle states can be stored.

7. Implementation

7.1 Authentication and Role Management

Registration collects the user's name, email, password, phone, address, city, coordinates, role and, for distributors and organizations, license or registration details and an uploaded document. Passwords are stored with `password_hash()` and verified with `password_verify()`. A successful login regenerates the session ID and stores the user's identity in the PHP session. Protected endpoints call a shared role check, and the dashboard hides tabs that are not allowed for the current role.

7.2 Admin Verification and Documents

New distributor and organization accounts remain unverified until an administrator approves them. The verification page lists pending accounts and shows a thumbnail of the uploaded document, which opens the full image in a new tab. Approve and reject actions are written to the audit log.

7.3 Medicine Inventory

Distributors add medicines with name, category, price, MRP, stock, batch number, expiry date, shelf life and image. Existing medicines can be edited. Removal uses a soft-delete flag so past orders and invoices remain intact. When stock reaches zero, the medicine is removed from the store and the distributor is notified. Expired medicines and out-of-stock medicines are also removed automatically when the store is loaded, and a low-stock notification is generated below the configured threshold.

7.4 Store, Cart and Geolocation

The store shows available medicines with distributor name, category, price, stock, expiry and shelf life. Users can search by name and filter by category, remaining shelf life and distributor. The cart is stored in the browser so it survives page reloads. Delivery location is captured through the browser geolocation API and stored in a cookie; if permission is denied, checkout proceeds with the delivery address alone.

7.5 Order Management

When an order is placed, the server validates each medicine, checks expiry and stock, deducts the ordered quantity, groups items by distributor and creates one order per distributor. The minimum order value is ₹500 per distributor. The order, its line items and its transaction are written inside a database transaction. Organizations can cancel an order while it is Pending or Pending Payment; the cancellation restores reserved stock and marks the transaction as Cancelled. Distributors can reject pending orders, which also restores stock.

7.6 Notifications, Reviews and Reports

Notifications are stored in the database with a type, text and read status. They are generated for new orders, payment confirmation, acceptance, rejection, cancellation, delivery, low stock and expiry. The navigation bar shows an unread-count badge refreshed every 60 seconds. Organizations can review a

delivered order once, with a rating and an optional comment. Reports show revenue, order counts, medicine counts and stock distribution, with CSV export. The audit log records sensitive actions with the user and timestamp.

7.7 Module-to-File Mapping

Module	Main Files
Authentication	`auth/login.php`, `logout.php`, `session.php`, `register.php`
Password reset	`forgot-password.html`, `auth/forgot.php`
Dashboard	`dashboard.html`, `js/app.js`
Medicine inventory	`api/medicines.php`
Store and cart	`index.html`, `js/app.js`
Orders	`api/orders.php`
Payments	`api/payment.php`
Notifications	`api/notifications.php`
Verification	`api/verify.php`
Reports and audit	`api/reports.php`, `api/audit.php`
Reviews and profile	`api/reviews.php`, `api/profile.php`
Map	`map.html`, `api/locations.php`

8. Order Lifecycle & Payments

8.1 Status Flow

```
Razorpay:  Pending Payment --(verified payment)--> Pending
Cash:      Pending (immediately)

Pending --(distributor accepts)--> Accepted --> Packed --> Shipped
        --> Out for Delivery --> Delivered

Pending / Pending Payment --(organization cancels)--> Cancelled
Pending --(distributor rejects)--> Cancelled
```

Stock is deducted when the order is created. If an order is cancelled or rejected, the stock is restored. When the last units of a medicine are ordered, the medicine is marked removed and the distributor is notified. An unpaid Razorpay order cannot be accepted by the distributor, so payment confirmation is enforced on the server.

8.2 Online Payments

The server creates a Razorpay order with the server-computed total, stores the gateway order reference in the transaction row and returns the public key and order details to the browser. The Razorpay checkout supports UPI, cards and net banking, and UPI can be pre-selected so that the UPI section opens first.

After payment, the browser sends the Razorpay order ID, payment ID and signature to the server. The server recomputes the HMAC-SHA256 signature with the secret key and compares it using a timing-safe comparison. Only a valid signature moves the orders from Pending Payment to Pending and marks the transaction as Completed.

8.3 Cash on Delivery

Cash orders are confirmed immediately with a Pending transaction. The distributor can mark the payment as received after delivery. Payment status is displayed separately from order status so that the two concerns are not confused.

9. Security

Security was treated as a cross-cutting concern rather than an add-on. The main measures are:

- passwords hashed with `password_hash()` and verified with `password_verify()`;
- prepared statements for every database query to prevent SQL injection;
- role checks on every protected endpoint, not only in the interface;
- session ID regeneration after login;
- HMAC-SHA256 verification of Razorpay payments with a timing-safe comparison;
- HMAC-signed, expiring password-reset tokens;
- generic responses for unknown email addresses to prevent account enumeration;
- a five-attempt limit on OTP verification;
- `.htaccess` denies for the configuration and log folders;
- PHP execution blocked inside the uploads folder;
- credentials stored in `config/secrets.php`, which is excluded from Git;
- user-uploaded license documents and logs excluded from Git;
- audit logging of sensitive actions.

The password-reset flow sends a 6-digit OTP through Supabase Auth, verifies it on the server, and issues a short-lived reset token. The new password is hashed before storage.

10. Performance Optimization

Because the database is hosted remotely, a cold database round trip costs hundreds of milliseconds to several seconds. The following optimizations were applied:

- **Short-lived file cache:** read-heavy GET endpoints cache JSON responses for 10–20 seconds. Every POST clears the cache, so writes are visible immediately.
- **Lazy database connection:** endpoints check the cache before opening a database connection, so cache hits avoid the database entirely.
- **Session lock release:** the PHP session is closed after reading the current user, allowing parallel requests.
- **Combined queries:** report statistics were reduced from four queries to one.
- **Prefetching:** the dashboard warms the APIs for all tabs after login, so tab switches hit the client or server cache.
- **Client-side cache:** the dashboard keeps GET responses for 30 seconds and the store keeps them longer.

Endpoint	Cold response	Cached response
Orders	~916 ms	5–32 ms
Medicines	~1403 ms	3–4 ms
Notifications	~797 ms	27–28 ms
Reports	~1585 ms	12–27 ms

The results show that caching and lazy connections remove most repeated database latency while short TTLs and write invalidation keep data fresh.

11. Testing & Results

Testing was performed at three levels: API tests using PHP and cURL, browser tests using Playwright, and a role-based smoke test that visited every page and tab.

#	Test	Expected Result	Status
1	Login with valid credentials	Session created and dashboard opens	Pass
2	Login with wrong password	401 Invalid email or password	Pass
3	Unverified account login	403 Account pending verification	Pass
4	Admin approves a distributor with a license image	Account verified; document visible	Pass
5	Distributor adds a medicine	Medicine appears in the store	Pass
6	Distributor removes a medicine	Medicine disappears; order history remains	Pass
7	Organization places a cash order	Order Pending; stock reduced	Pass
8	Organization places a Razorpay order	Pending Payment; distributor cannot accept	Pass
9	Razorpay payment with a valid signature	Order Pending; transaction Completed	Pass
10	Razorpay payment with an invalid signature	Payment rejected	Pass
11	Organization cancels a pending order	Cancelled; stock restored; transaction Cancelled	Pass
12	Organization tries to cancel an accepted order	409 error	Pass
13	Ordering the last units of a medicine	Medicine removed; distributor notified	Pass
14	Expired medicine in the store	Medicine removed; distributor notified	Pass
15	Unread notifications	Badge shows the count	Pass
16	Review on a delivered order	Review saved; second review rejected	Pass
17	Forgot password with a registered email	OTP generated and verified	Pass
18	Password reset with a valid token	Password updated; login works	Pass
19	Reuse of a used OTP	Rejected	Pass
20	Tab access outside the user's role	Tab hidden; API returns 403	Pass

All API and UI test suites passed. The smoke test rendered every role's tabs with zero JavaScript errors; the only console messages were expected 401 responses for guests. Test data was removed after the runs.

12. Limitations & Future Work

12.1 Limitations

- Email and SMS delivery require external configuration; email is disabled by default and SMS is a stub.
- Razorpay is used in test mode; live payments require live keys and compliance checks.
- There is no integration with courier or delivery-partner APIs.
- Geolocation depends on browser permission and may be inaccurate.
- The application is a responsive web app, not a native mobile app.
- The hosted database adds network latency to cold requests.
- Admin verification is manual, and the interface is available in English only.

12.2 Future Work

- Enable live Razorpay payments and configure SMTP for real email delivery.
- Integrate an SMS gateway such as Twilio or MSG91.
- Add delivery-partner tracking and route optimization.
- Add barcode or QR scanning for medicine batches.
- Use OCR to assist license verification.
- Add demand forecasting and early expiry alerts.
- Build a mobile application for distributors and field staff.
- Add multi-language support, accessibility improvements, backups and monitoring.

13. Conclusion & References

13.1 Conclusion

MediShare demonstrates how a focused web application can reduce medicine wastage and improve access for social organizations. The project covers the complete operational cycle: registration, verification, inventory, store, cart, payment, order tracking, cancellation, notifications, reviews, reports and password recovery.

The implementation emphasizes correctness at trust boundaries: verified accounts, server-side payment verification, transactional stock handling, role-based authorization and audit logging. Caching and lazy database access keep the interface responsive despite a hosted database. The system is ready for extension with live payment credentials, real email and SMS delivery, and delivery-partner integration.

13.2 References

1. PHP Documentation — https://www.php.net/docs.php
2. PostgreSQL Documentation — https://www.postgresql.org/docs/
3. Supabase Documentation — https://supabase.com/docs
4. Razorpay Payments Documentation — https://razorpay.com/docs/
5. Bootstrap 5 Documentation — https://getbootstrap.com/docs/5.3/
6. MDN Web Docs — https://developer.mozilla.org/
7. OWASP Application Security Verification Standard — https://owasp.org/www-project-application-security-verification-standard/
8. Playwright Documentation — https://playwright.dev/
9. Leaflet Documentation — https://leafletjs.com/
10. OpenStreetMap — https://www.openstreetmap.org/

Appendix — Setup & API Reference

A. Setup Instructions

1. Install XAMPP with PHP 8.2 and enable the `pdo_pgsql` and `curl` extensions.
2. Import `database.sql` into a PostgreSQL or Supabase database.
3. Copy `config/secrets.example.php` to `config/secrets.php` and fill in the database, Razorpay and Supabase values.
4. Place the project under the Apache document root and open `index.html`.
5. For password-reset email, add `{{ .Token }}` to the Magic Link template in Supabase Authentication → Email Templates.

B. Seed Accounts

All seed accounts use the password `admin123`.

Role	Email
Administrator	admin@medshare.com
Distributor	medplus@medshare.com
Distributor	pharmalink@medshare.com
Organization	hope@medshare.com
Organization	citycare@medshare.com

C. Configuration Keys

All values are stored in `config/secrets.php`, which is excluded from version control.

Constant	Purpose
`DB_DSN`, `DB_USER`, `DB_PASS`	PostgreSQL connection
`RAZORPAY_KEY_ID`, `RAZORPAY_KEY_SECRET`	Payment gateway credentials
`SUPABASE_URL`, `SUPABASE_ANON_KEY`	Supabase Auth for password-reset OTP
`RESET_SECRET`	Signing key for reset tokens

D. API Endpoints

Endpoint	Purpose
`auth/login.php`, `auth/logout.php`, `auth/session.php`	Authentication
`auth/register.php`	Registration with document upload
`auth/forgot.php`	Password-reset OTP request, verification and reset
`api/medicines.php`	Medicine list, details, add, edit, remove
`api/orders.php`	Order creation, status actions, cancellation, list
`api/payment.php`	Razorpay order creation and verification
`api/notifications.php`	List and mark notifications as read
`api/verify.php`	Admin approval of accounts
`api/distributors.php`	Distributor profiles and admin list
`api/reviews.php`	Reviews for delivered orders
`api/reports.php`	Statistics and CSV export
`api/audit.php`	Audit log
`api/profile.php`	Profile update
`api/locations.php`	Verified locations for the map

End of Report